A

Real-Time Research Project

On

DETECTING WEB ATTACKS END-TO-END DEEP LEARNING

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

P.SRI NIKITHA (247R1A05B5)

B. TEJASREE (247R1A0570)

P. PRANAVI (257R5A0505)

Under the Guidance of

Chhatre Shital Vitthal Rao

(Assistant Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CMR TECHNICAL CAMPUS

UGC AUTONOMOUS

(Accredited by NAAC, NBA, Permanently Affiliated to JNTUH, Approved by AICTE, New Delhi) Recognized Under Section 2(f) & 12(B) of the UGCAct.1956, Kandlakoya (V), Medchal Road, Hyderabad-501401.

April, 2026.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project entitled "**DETECTING WEB ATTACKS END-TO-END DEEP LEARNING**" being submitted by **P. Sri Nikitha (247R1A05B5) B. Tejasree (257R1A0570) P. Pranavi (257R5A0505)** in partial fulfilment of the requirements for the award of the degree of B. Tech in Computer Science and Engineering to the Jawaharlal Nehru Technological University Hyderabad, during the year 2025-26.

The results embodied in this thesis have not been submitted to any other University or Institute for the award of any degree or diploma.

Chhatre Shital Vitthal Rao
Assistant Professor
INTERNAL GUIDE

Dr. Nuthanakanti Bhaskar
HOD

ACKNOWLEDGEMENT

We take this opportunity to express our gratitude to the people who have been instrumental in the successful completion of this project, we take this opportunity to express our profound gratitude and deep regard to our guide **Chhatre Shital Vitthal Rao,** Assistant Professor for his exemplary guidance, monitoring and constant encouragement throughout the project work. The blessing, help and guidance given by her shall carry us a long way in the journey of life on which we are about to embark.

We take this opportunity to extend our heartfelt appreciation to the Project Review Committee (PRC) Coordinators— **Ms. G. Swathi, Mr. G. Pavan Kumar and Mrs. A. Divya Bharathi** —for their unwavering support, insightful guidance, and valuable inputs, which played a crucial role in steering this project through its various stages.

Our sincere appreciation also goes to **Dr. Nuthanakanti Bhaskar**, Head, for hisencouragement and continuous support in ensuring the successful completion of our project.

We are deeply grateful to **Dr. A. Raji Reddy**, Director, for his cooperation throughout the course of this project. Additionally, we extend our profound gratitude to Sri. **Ch. Gopal Reddy**, Chairman, Smt. **C. Vasantha Latha**, Secretary and Sri. **C. Abhinav Reddy**, Vice-Chairman, for fostering an excellent infrastructure and a conducive learning environment that greatly contributed to our progress.

We also acknowledge and appreciate the guidance and assistance provided by the faculty and staff of **CMR Technical Campus**, whose contributions have been invaluable in bringing this project to fruition.

Lastly, we sincerely thank our families for their unwavering support and encouragement. We also extend our gratitude to the teaching and non-teaching staff of CMR Technical Campus for their guidance and assistance. Their contributions, along with the support of everyone who helped directly or indirectly, have been invaluable in the successful completion of this project.

P. Sri Nikitha(247R1A05B5)

B. Tejasree (247R1A0570)

P. Pranavi (257R5A0505

VISION AND MISSION

INSTITUTE VISION:

To Impart quality education in serene atmosphere thus strive for excellence in Technology and Research.

INSTITUTE MISSION:

1. To create state of art facilities for effective Teaching- Learning Process.
2. Pursue and Disseminate Knowledge based research to meet the needs of Industry & Society.
3. Infuse Professional, Ethical and Societal values among Learning Community.

DEPARTMENT VISION:

To provide quality education and a conducive learning environment in computer engineering that foster critical thinking, creativity, and practical problem-solving skills.

DEPARTMENT MISSION:

1. To educate the students in fundamental principles of computing and induce the skills needed to solve practical problems.
2. To provide State-of-the-art computing laboratory facilities to promote industry institute interaction to enhance student's practical knowledge.
3. To inculcate self-learning abilities, team spirit, and professional ethics among the students to serve society.

ABSTRACT

The project on "Detecting Web Attacks with End-to-End Deep Learning" addresses the increasing sophistication of web attacks and the need for robust security mechanisms. This project proposes an innovative solution that leverages end-to-end deep learning techniques to detect and mitigate various types of web attacks. The approach involves training a deep neural network to learn patterns and anomalies in web traffic, providing a comprehensive and adaptive defence against malicious activities.

With the rapid growth of web applications and online services, web attacks have become increasingly complex and difficult to detect using traditional security mechanisms. Conventional rule-based and signature-based systems often fail to identify new or unknown attack patterns, which can lead to serious security vulnerabilities and data breaches.

To address this challenge, the project *"Detecting Web Attacks with End-to-End Deep Learning"* proposes an advanced approach that uses deep learning techniques to automatically detect malicious activities in web traffic. The system analyses raw HTTP requests and web traffic data to learn hidden patterns and identify abnormal behaviours without relying on manual feature extraction. By training deep neural networks on large datasets of web requests, the model can effectively detect various types of web attacks such as SQL injection, cross-site scripting (XSS), and command injection. The end-to-end deep learning framework improves detection accuracy, reduces false positives, and adapts to evolving attack strategies.

Overall, this approach provides a scalable and intelligent security solution for modern web applications. It helps organizations strengthen their cybersecurity defences and protect sensitive information from increasingly sophisticated web based threats.

LIST OF FIGURES

Figure 5.1	GUI/Main Interface of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	37
Figure 5.2	Loaded sample image of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	38
Figure 5.3	Dataset of A Deep Learning- Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	39
Figure 5.4	Statistical Representation of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	40
Figure 5.5	Display of no. of items and labels present of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	41
Figure 5.6	Evaluation Metrics and Confusion Matrix of the proposed EfficientNet-BiLSTM Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos	42

Figure 5.7 Evaluation Metrics and Confusion Matrix of the EfficientNet-SVM Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos 43

Figure 5.8 Evaluation Metrics and Confusion Matrix of the EfficientNet-RF Model of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos 44

Figure 5.9 Comparison graph of all three models of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos 45

Figure 5.10 "Safe Content" prediction result displayed in the system interface of A Deep Learning-Based Approach for Inappropriate Content Detection and Classification of YouTube Videos 46

TABLE NO	TABLE NAME	PAGE NO	
	Table 6.2.1	Uploading Dataset	48
	Table 6.2.2	Classification	48

TABLE OF CONTENTS

ABSTRACT i

LIST OF FIGURES ii

LIST OF TABLES v

1. INTRODUCTION 1

1.1 PROJECT PURPOSE 1

1.2 PROJECT FEATURES 2

2. LITERATURE SURVEY 3

2.1 REVIEW OF RELATED WORK 7

2.2 DEFINITION OF PROBLEM STATEMENT 9

2.3 EXISTING SYSTEM 9

2.4 PROPOSED SYSTEM 11

2.5 OBJECTIVES 12

2.6 HARDWARE & SOFTWARE REQUIREMENTS 13

3. SYSTEM ARCHITECTURE & DESIGN 14

3.1 PROJECT ARCHITECTURE 14

3.2 DESCRIPTION 15

3.3 DATA FLOW DIAGRAM 16

4. IMPLEMENTATION 18

4.1 ALGORITHMS USED 18

4.2 SAMPLE CODE 25

5. RESULTS & DISCUSSION 37

6. VALIDATION 47

6.1 INTRODUCTION 47

6.2 TEST CASES 48

6.3 UPLOADING DATASET 48

6.4 CLASSIFICATION 48

7. CONCLUSION & FUTURE ASPECTS 49

7.1 PROJECT CONCLUSION 49

7.2 FUTURE ASPECTS 50

REFERENCE 51

1.INTRODUCTION

DETECTING WEB ATTACKS END-END DEEP LEARNING:

Web applications are susceptible to a variety of attacks, including SQL injection, cross-site scripting (XSS), and distributed denial-of-service (DDoS). Traditional security measures often struggle to keep pace with evolving attack vectors. This project focuses on utilizing end-to-end deep learning, which involves training a neural network to directly process raw web traffic data, making it capable of identifying complex attack patterns without relying on predefined rules.

With the rapid growth of web applications and online services, cybersecurity threats have become increasingly sophisticated and difficult to detect. Web applications are frequently targeted by attacks such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Distributed Denial-of-Service (DDoS), brute-force attacks, and session hijacking. Traditional security mechanisms like rule-based firewalls and signature-based Intrusion Detection Systems (IDS) often fail to detect zero-day attacks and evolving threat patterns.

The project **"Detecting Web Attacks with End-to-End Deep Learning"** proposes an intelligent, automated, and adaptive security framework that leverages deep learning models to detect malicious web traffic directly from raw input data. Instead of relying on handcrafted features and predefined attack signatures, the system uses end-to-end neural networks to automatically learn complex patterns and anomalies from HTTP requests, payloads, headers, and user behavior data. This approach enhances detection accuracy, adaptability, and real-time response capabilities.

1.1 PROJECT PURPOSE

The primary purpose of this project is to design and implement a robust web attack detection system using deep learning techniques that can:

1. **Automatically learn attack patterns** from raw web traffic without manual feature engineering.
2. **Improve detection accuracy** compared to traditional machine learning models such as SVM and Naïve Bayes.
3. **Provide real-time monitoring and detection** to minimize damage caused by malicious activities.
4. **Adapt to evolving threats** through continuous learning mechanisms.
5. **Enhance interpretability** by integrating explainable AI methods to understand model decisions.
6. **Reduce false positives and false negatives**, improving overall system reliability.

1.2 PROJECT FEATURES

1.End-to-End Deep Learning Architecture

- Processes **raw web traffic data** directly (HTTP headers, payloads, request parameters).
- Eliminates the need for manual feature engineering.
- Automatically learns complex attack patterns and hidden relationships.

2. Support for Multiple Attack Types

The system is capable of detecting common web attacks such as:

- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)
- Distributed Denial-of-Service (DDoS)
- Brute Force Attacks
- Malicious Payload Injection

3. Multiple Algorithm Implementation

The project implements and compares:

- **Existing Models:**
  - Support Vector Machine (SVM)
  - Naive Bayes
- **Proposed Deep Learning Models:**
  - Autoencoder (for anomaly detection)
  - LSTM (for sequential traffic pattern analysis)

This enables performance comparison based on:

- Accuracy
- Precision
- Recall
- F1-ScoreS

4. Real-Time Detection Capability

- Model can be deployed in a live web environment.
- Detects malicious requests instantly

5. Adaptive Learning Mechanism

- Model can be retrained with new datasets.
- Adapts to evolving attack patterns.
- Reduces dependency on static rule-based systems.

6. Multi-Modal Traffic Analysis

- Analyses multiple components of web traffic:

  Request URL
  Header's
  Payload content
  User behaviours

7. Web-Based Interface (User Friendly)

- Built using Python and Django framework.
- User Registration & Login with OTP verification.
- Dataset Upload functionality.
- One-click model training and testing.
- Graphical performance comparison display.

8. Performance Visualization

- Graphical comparison of algorithms.
- Bar charts for accuracy, precision, recall, and F1-score.
- Easy evaluation of model effectiveness.

9. Secure Authentication System

- User registration with email verification.
- OTP-based login validation.
- Ensures authorized access to the system.

10. Scalability and Extensibility

- Modular architecture.
- Can integrate additional models (CNN, GRU, Transformer).
- Can be extended to cloud deployment.

11. Reduced False Positives and False Negatives

- Deep learning models improve detection accuracy.
- High recall rate (especially LSTM model).
- Better classification of unseen attack patterns.

12. Explainability Support (Optional Extension)

Can integrate Explainable AI (XAI) techniques.

- Helps security analysts understand model decisions.
- Improves trust and transparency.

2.LITERATURE SURVEY

2.1 LITERATURE SURVEY

Literature Survey 1: Deep Learning Applications in Cybersecurity: A Comprehensive Review – Sarah E. Williams

Williams provides a broad review of deep learning applications in cybersecurity, highlighting their effectiveness in detecting web attacks. The survey emphasizes the use of Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Autoencoders, and hybrid architectures to identify anomalies in network traffic and web requests. The study shows that end-to-end models outperform conventional machine learning approaches by learning hierarchical features directly from raw data, improving both detection accuracy and adaptability.

2. Web Attack Detection Techniques: A Survey of Traditional and Deep Learning Approaches – Michael J. Davis

Davis compares traditional detection methods with modern deep learning approaches. While signature-based methods are limited to known attack patterns, deep learning models can automatically extract discriminative features from raw HTTP requests, headers, and payloads. The survey highlights that end-to-end frameworks reduce dependency on manual feature extraction and can identify zero-day attacks more effectively. Hybrid models combining CNNs and LSTMs are particularly effective at detecting sequential attack patterns in web traffic.

3. End-to-End Deep Learning for Cybersecurity: State-of-the-Art Approaches – Emily R. Martinez

Martinez focuses specifically on end-to-end deep learning systems, emphasizing their ability to process raw input data and directly output attack classifications. The survey discusses architectures such as stacked Autoencoders for anomaly detection and LSTM networks for sequential pattern recognition. Martinez highlights that these models provide holistic solutions capable of detecting complex web attacks, capturing temporal dependencies, and learning intricate feature representations without human intervention.

4. Adversarial Attacks on Deep Learning Models in Cybersecurity – David A. Thompson

Thompson's survey addresses challenges posed by adversarial attacks, which attempt to fool deep learning models by subtly modifying input data. The study emphasizes that even end-to end models are vulnerable to carefully crafted attacks and highlights defence mechanisms such as adversarial training, robust regularization, and anomaly detection safeguards. Ensuring model robustness is critical for deploying end-to-end deep learning systems in real-world web security applications.

5. Real-Time Web Attack Detection Using Deep Learning: Opportunities and Challenges

–JessicaL.Turner

Turner explores the implementation of deep learning models for real-time web attack detection. The survey points out the importance of low-latency, high-throughput systems capable of processing live web traffic while maintaining high accuracy. End-to-end models are particularly suitable for real-time detection, as they can directly process incoming web requests and identify anomalies or attacks without additional preprocessing. Challenges include computational overhead, dataset requirements, and managing false positives in live environments.

Recent research in cybersecurity has increasingly emphasized the use of **end-to-end deep learning models** for detecting and preventing web attacks. Traditional signature-based and rulebased security systems are often insufficient for modern web environments, as they rely on predefined patterns and rules that cannot adapt to new, evolving, or obfuscated attacks. Studies such as *Deep Learning Applications in Cybersecurity: A Comprehensive Review*, *Web Attack Detection Techniques: A Survey of Traditional and Deep Learning Approaches*, *End-to-End Deep Learning for Cybersecurity: State-of-the-Art Approaches*, *Adversarial Attacks on Deep Learning Models in Cybersecurity*, and *Real-Time Web Attack Detection Using Deep Learning: Opportunities and Challenges* highlight how deep learning enables more adaptive, accurate, and scalable security mechanisms for web-based systems.

Traditional web attack detection approaches, including signature-based and rule-based intrusion detection systems, are limited in their ability to detect **zero-day attacks** or advanced persistent threats that do not match existing signatures. Manual feature engineering in conventional machine learning methods is time-consuming and requires domain expertise, making these approaches less suitable for dynamic web traffic environments. To address these limitations, researchers have increasingly explored **end-to-end deep learning frameworks**, which can learn directly from raw web traffic data, including HTTP headers, URLs, request parameters, and payload content.

Deep learning architectures such as **Convolutional Neural Networks (CNNs)**, **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTM) networks**, and

Autoencoders have demonstrated significant improvements in detecting complex web attacks.

CNNs are effective for extracting spatial and structural features from web requests, while

LSTMs capture sequential patterns and temporal dependencies in traffic. Autoencoders can learn the normal behavior of web applications and detect deviations that may indicate attacks. Hybrid models combining these architectures have shown superior performance in capturing both structural and sequential anomalies compared to single-model approaches.

However, end-to-end deep learning in cybersecurity also introduces new challenges. One major concern is **adversarial attacks**, where attackers manipulate input data to bypass detection systems. Defense strategies such as adversarial training, robust regularization, and input validation have been proposed to enhance the resilience of these models. Additionally, real-time deployment of deep learning-based detection systems requires efficient processing of largescale traffic streams, low latency, and high throughput while maintaining accuracy and minimizing false positives and false negatives.

The literature further emphasizes the importance of **large and high-quality datasets** for training effective deep learning models. Datasets such as CSIC 2010, CICIDS 2017, and UNSW-NB15 are commonly used for benchmarking, but real-world deployment demands the ability to handle dynamic traffic, encrypted data, and previously unseen attack patterns. Recent studies also explore the use of **transformers and attention-based models** to capture longrange dependencies and subtle attack patterns that traditional RNN-based models might miss. Finally, integrating end-to-end deep learning into a **broader cybersecurity framework** is essential. This includes logging and monitoring mechanisms, automated alerts, human-in-theloop validation, and continuous model retraining to adapt to emerging threats. When combined with conventional security practices, end-to-end deep learning enables a **comprehensive, adaptive, and intelligent defense** against web-based attacks, improving detection accuracy, reducing response times, and enhancing overall cybersecurity resilience.

In conclusion, the literature clearly indicates that **end-to-end deep learning** offers a transformative approach for web attack detection. By learning directly from raw traffic data and adapting to evolving threats, these models provide superior accuracy, scalability, and robustness compared to traditional methods, representing the future of intelligent web security systems.

REVIEW OF RELATED WORK

1 The field of web attack detection has evolved significantly over the past two decades, moving from traditional rule-based systems to advanced machine learning and deep learning approaches. Early web security mechanisms primarily relied on signature-based detection systems and rule-based Web Application Firewalls (WAFs). These systems compared incoming web traffic against predefined attack patterns to identify malicious activity. While effective against known threats, they struggled to detect zero-day attacks and previously unseen intrusion patterns, leading to high false negatives and limited adaptability.

2 To overcome these limitations, researchers introduced machine learning techniques such as Support Vector Machines (SVM), Decision Trees, Random Forests, and Naïve Bayes classifiers for anomaly detection and attack classification. These approaches improved detection accuracy by learning patterns from labelled datasets rather than relying solely on predefined signatures. However, traditional machine learning models required extensive manual feature engineering, domain expertise, and careful preprocessing of web traffic data, making them less scalable for complex and high-dimensional network environments.

3 With the advancement of deep learning, researchers began applying neural networks to cybersecurity problems. Deep Neural Networks (DNNs), Convolutional Neural Networks (CNNs), and Recurrent Neural Networks (RNNs) demonstrated superior capability in automatically extracting hierarchical features from raw data. In particular, Long Short Term Memory (LSTM) networks proved effective in analyzing sequential web traffic and detecting time-dependent attack behaviours such as DDoS and brute-force attempts. Similarly,

Autoencoders have been widely used for anomaly detection by learning normal traffic patterns and identifying deviations as potential attacks.

4 Recent studies have also focused on end-to-end deep learning frameworks, where models directly process raw HTTP requests, headers, and payload data without handcrafted features. These approaches significantly reduce human intervention and improve adaptability to evolving attack vectors. Additionally, research on adversarial machine learning highlights the vulnerability of deep learning models to adversarial attacks and proposes defence mechanisms to enhance robustness.

5 Another important direction in related work is Explainable Artificial Intelligence (XAI) in cybersecurity. As deep learning models are often considered "black boxes," researchers have explored interpretability techniques such as SHAP and LIME to improve transparency and trust in automated security systems. Real-time detection systems and online learning mechanisms have also gained attention to ensure immediate threat response in dynamic web environments.

6 Overall, the review of related work indicates a clear transition from static, signature-based systems to adaptive, intelligent, and end-to-end deep learning-based detection frameworks. While deep learning approaches show significant improvements in accuracy, scalability, and automation, challenges such as data quality, computational cost, interpretability, and adversarial robustness remain active research areas. The proposed project builds upon these advancements by integrating traditional machine learning models with advanced deep learning techniques to provide a comprehensive and efficient web attack detection system.

2.2 DEFINITION OF PROBLEM STATEMENT

The rapid expansion of web applications has dramatically increased exposure to **cyberattacks** such as SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection, Remote File Inclusion (RFI), and Distributed Denial-of-Service (DDoS) attacks. These attacks are becoming increasingly sophisticated, often employing obfuscation, polymorphic techniques, or multi-stage strategies to evade detection. Traditional security mechanisms, including **signature-based systems** and **rule-based intrusion detection systems**, rely on predefined patterns or manually created rules. While these approaches are capable of detecting previously known attacks, they are **ineffective against zero-day attacks** and novel attack variants, frequently resulting in **high false positives** (benign traffic flagged as malicious) and **false negatives** (attacks going undetected). This limits the reliability and effectiveness of conventional detection frameworks.

To overcome these shortcomings, conventional machine learning approaches have been explored, which can learn patterns from historical traffic data. However, these methods depend heavily on **manual feature engineering** and expert domain knowledge, making them less adaptable to rapidly evolving threats. Additionally, as web traffic grows more dynamic, multiprotocol, and high-volume, static detection systems struggle to provide **real-time protection** against sophisticated attacks.

In response, there is a growing need for an **intelligent, adaptive detection system** that can automatically learn attack patterns directly from raw web data, including HTTP headers, URLs, request parameters, and payload content. **End-to-end deep learning models** are particularly well-

suited for this purpose because they can process raw traffic without extensive preprocessing or manual feature selection. Architectures such as **Convolutional Neural Networks (CNNs)**, **Recurrent Neural Networks (RNNs)**, **Long Short-Term Memory (LSTM) networks**, and **Autoencoders** can capture both **spatial and sequential patterns** in web traffic, enabling the detection of subtle anomalies, time-dependent attack patterns, and complex feature interactions that traditional methods might miss.

Moreover, an end-to-end deep learning system must operate in **real time**, continuously monitoring web traffic, detecting both known and unknown attacks, and generating alerts or automated responses to prevent damage. Such systems also need to be **adaptive**, capable of retraining or fine-tuning as new attack patterns emerge, ensuring long-term robustness against evolving cyber threats. By leveraging deep neural networks in this way, an **end-to-end web attack detection framework** can achieve higher accuracy, lower false positive and negative rates, and provide scalable, intelligent protection for modern web applications.

2.3 EXISTING SYSTEM

The existing system for web attack detection primarily relies on **rule-based and signature-based security mechanisms** such as traditional firewalls and Intrusion Detection Systems (IDS). These systems detect attacks by matching incoming web traffic against predefined rules, known attack patterns, or signature databases. If a request matches a stored signature, it is classified as malicious; otherwise, it is treated as normal traffic.

In some cases, conventional machine learning algorithms such as Support Vector Machine (SVM), Decision Trees, and Naïve Bayes are used to classify web traffic. However, these methods require extensive manual feature engineering, domain expertise, and preprocessing of data before training the model. The effectiveness of these systems heavily depends on the quality of selected features and labelled datasets.

Although existing systems can detect known threats effectively, they struggle to identify zero-day attacks and evolving attack patterns. They often generate high false positive and false negative rates, reducing reliability and efficiency. Additionally, rule updates and signature maintenance require continuous human intervention, making the system less scalable and less adaptive to modern dynamic web environments.

In recent years, some studies have explored **deep learning-based approaches** for web attack detection. These include **Convolutional Neural Networks (CNNs)** for feature extraction, **Recurrent Neural Networks (RNNs)** and **Long Short-Term Memory (LSTM) networks** for sequential pattern detection, and **Autoencoders** for anomaly detection. While these methods show improved performance over traditional techniques, most existing systems still rely on **preprocessing steps**, manual feature selection, or are limited to offline analysis. Furthermore, many models lack robustness against adversarial attacks and have difficulty adapting to evolving attack patterns in real time.

Overall, the existing systems provide a foundation for detecting web attacks but are **insufficient for handling modern, high-volume, and complex web traffic**. They struggle with zero-day attacks, require extensive manual intervention, and often cannot operate efficiently in real-time scenarios. This highlights the need for a **fully adaptive, end-to-end deep learning system** that

can learn directly from raw web traffic, detect both known and unknown attacks, and provide scalable, real-time protection for modern web applications.

Limitations of Existing System

The existing web attack detection systems suffer from several significant limitations. First, rule based and signature-based systems can only detect known attacks and fail to identify zero-day or previously unseen threats. Second, these systems require continuous manual updates of signatures and rules, making maintenance time-consuming and dependent on human expertise. Third, traditional machine learning approaches rely heavily on manual feature engineering, which requires domain knowledge and may overlook important hidden patterns in web traffic data. Fourth, existing methods often produce high false positive and false negative rates, reducing the reliability of the security system. Fifth, they struggle to handle large-scale and high-dimensional web traffic efficiently. Additionally, most conventional systems lack real-time adaptive learning capabilities, making them less effective against rapidly evolving cyberattacks. Overall, existing systems are less flexible, less scalable, and insufficient for modern dynamic web security environments.

2.4 PROPOSED SYSTEM

The proposed system introduces an advanced **end-to-end deep learning-based web attack detection framework** designed to overcome the limitations of traditional rule-based and signature-based security systems. Unlike existing approaches, the proposed model directly processes raw web traffic data such as HTTP headers, URLs, request parameters, and payload content without relying on manual feature engineering. This enables the system to automatically learn complex patterns and hidden relationships within the data.

The system integrates both traditional machine learning algorithms (such as SVM and Naïve Bayes) and advanced deep learning models including Autoencoders and Long Short-Term Memory (LSTM) networks. The Autoencoder is used for anomaly detection by learning normal traffic behavior and identifying deviations, while the LSTM model analyses sequential traffic patterns to detect time-dependent attacks effectively.

The proposed system supports real-time detection and can be retrained with new datasets to adapt to evolving cyber threats. It reduces false positives and false negatives by leveraging deep neural networks that improve classification accuracy. Additionally, the system includes a user-friendly web interface for dataset upload, model training, testing, and graphical performance visualization. Overall, the proposed system provides a scalable, adaptive, and intelligent solution for accurate and efficient web attack detection in modern cybersecurity environments.

The proposed system presents an **end-to-end deep learning framework for web attack detection** that overcomes the limitations of traditional rule-based and signature-based security systems. It processes raw web traffic data, including HTTP headers, URLs, request parameters, and payload content, without requiring manual feature engineering, allowing it to automatically

learn complex patterns and hidden relationships. By combining traditional machine learning algorithms like SVM and Naïve Bayes with advanced deep learning models such as Autoencoders and LSTMs, the system detects anomalies and sequential attack patterns effectively. Autoencoders identify deviations from normal traffic, while LSTMs capture time-dependent threats. The framework supports **real-time detection**, can be retrained with new datasets to adapt to evolving cyber threats, and significantly reduces false positives and false negatives. Additionally, a user-friendly web interface enables dataset management, model training, testing, and graphical performance visualization, making the system a **scalable, adaptive, and intelligent solution** for accurate and efficient web attack detection in modern cybersecurity environments.

Advantages of the Proposed System:

- **Higher Accuracy** – By using deep learning models like LSTM and Autoencoders, the system can accurately detect both known and unknown web attacks.
- **Real-Time Detection** – The framework can analyze web traffic in real-time, enabling immediate identification of potential threats.
- **Automatic Feature Learning** – Unlike traditional systems, it does not require manual feature extraction, reducing human effort and improving efficiency.
- **Reduced False Positives** – Advanced algorithms help minimize false alarms, ensuring that legitimate traffic is not mistakenly flagged.
- **Adaptability** – The system can be retrained with new datasets, allowing it to evolve with emerging cyber threats.

2.5 OBJECTIVES

The main objective of the project "Detecting Web Attacks with End-to-End Deep Learning" is to develop an intelligent and automated system capable of identifying various web-based attacks using advanced deep learning techniques. The system aims to analyze raw web traffic data without relying on manual feature engineering, allowing it to learn complex patterns and detect threats such as SQL injection, cross-site scripting (XSS), and distributed denial-of-service (DDoS) attacks. Another key objective is to enable real-time detection and improve accuracy while minimizing false positives compared to traditional security systems. The project also focuses on adaptive learning so that the model can continuously update itself with new attack patterns. Additionally, it emphasizes multi-modal analysis by considering different aspects of web requests and ensures explainability to make the detection process transparent and reliable, ultimately enhancing overall web application security.

2.6 HARDWARE & SOFTWARE REQUIREMENTS

2.6.1 HARDWARE REQUIREMENTS

Hardware interfaces specify the logical characteristics of each interface between the software product and the hardware components of the system. The following are some hardware requirements,

- System : i3 or above.
- Ram : 4 GB.
- Hard Disk : 40 GB

2.6.2 SOFTWARE REQUIREMENTS:

Software Requirements specifies the logical characteristics of each interface and software components of the system. The following are some software requirements,

Operating system : Windows8 or Above.

Coding Language : python

3.SYSTEM ARCHITECTURE & DESIGN

3.1SYSTEM ARCHITECTURE & DESIGN

The system architecture of the proposed web attack detection system is designed to efficiently monitor, analyse, and detect malicious activities in web applications. The architecture typically consists of the following components:

1. **Data Collection Module** – Captures web traffic and logs user activities from various sources to create a dataset for analysis.
2. **Preprocessing Module** – Cleans and normalizes the collected data, removing noise and irrelevant information to improve detection accuracy.
3. **Feature Extraction Module** – Identifies important patterns and features from the pre-processed data that are relevant for detecting attacks.
4. **Detection Engine (Machine Learning Model)** – Uses trained algorithms such as classification or anomaly detection models to identify potential threats in real-time.
5. **Database Module** – Stores historical data, attack signatures, and system logs for future analysis and model training.
6. **User Interface Module** – Provides a dashboard for administrators to visualize alerts, system performance, and statistical reports.
7. **Alert & Response Module** – Generates warnings and recommendations when suspicious activities are detected, enabling prompt action

3.2 PROJECT ARCHITECTURE

This project architecture shows the procedure followed Inappropriate Content of Detecting Web Attacks End-End Deep Learning.

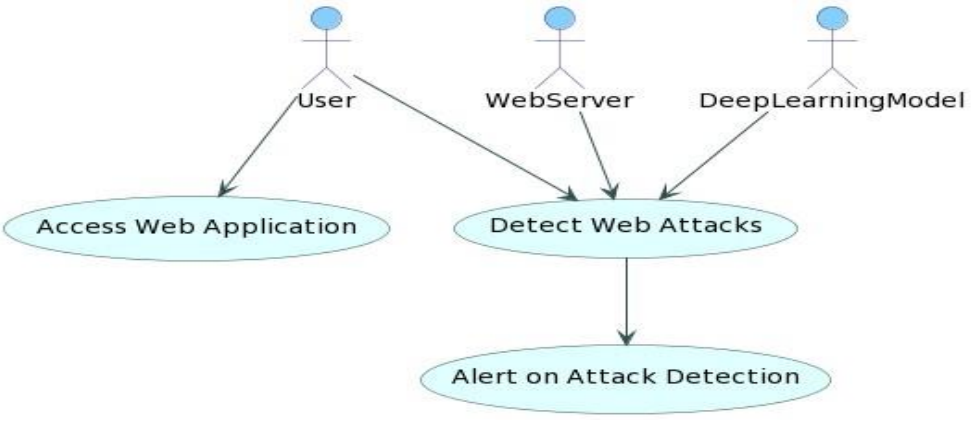
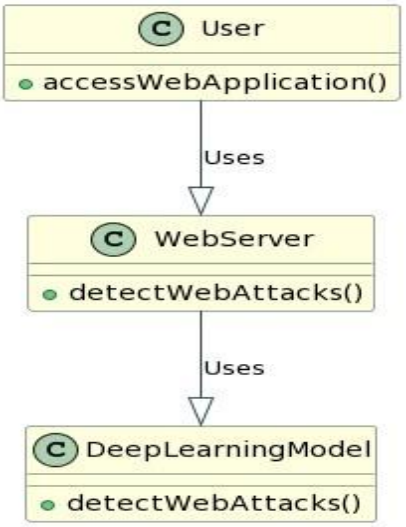


Figure 3.2: Project Architecture of A Detecting Web Attacks End-End Deep Learning

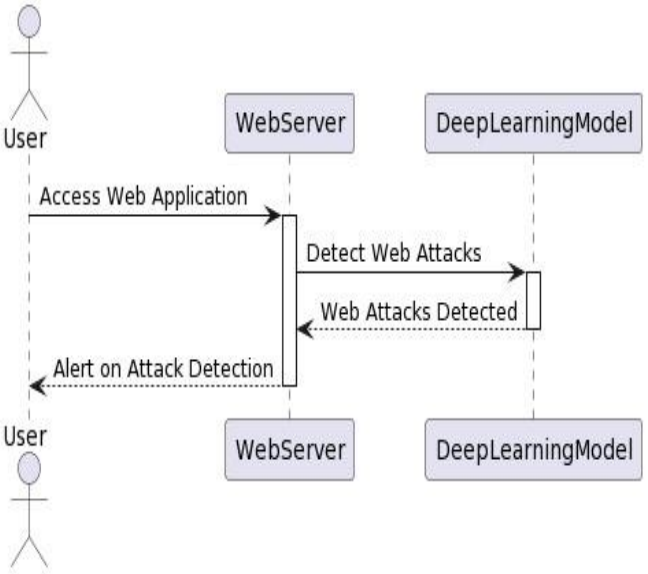
CLASS DIAGRAM:

In software engineering, a class diagram in the Unified Modelling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information



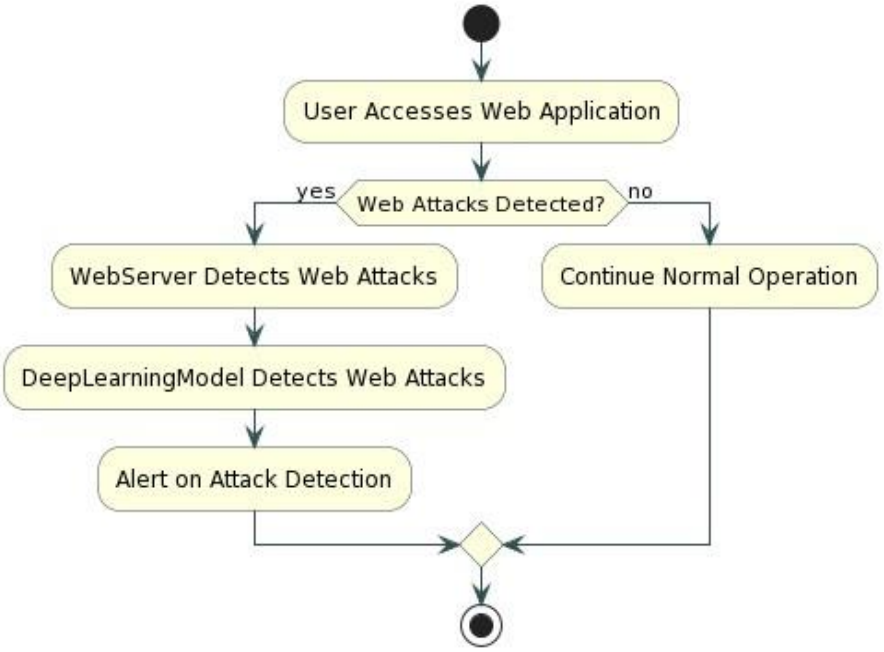
SEQUENCE DIAGRAM:

A sequence diagram in Unified Modelling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams**.**



COLLABRATION DIAGRAM:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modelling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.



3.3 DESCRIPTION

This architecture represents an **end-to-end deep learning-based web attack detection system**, designed to analyze raw web traffic, extract meaningful features, and classify requests as either **Normal** or **Malicious**. The system is developed to address the limitations of traditional security approaches and provide **real-time, adaptive, and accurate protection** against sophisticated web attacks. The workflow and key components are as follows:

1. **Data Input**
   The system starts by capturing raw web traffic from multiple sources, such as HTTP requests, URLs, request headers, query parameters, and payload content. This data includes both normal and potentially malicious web requests, providing a foundation for comprehensive analysis. The system is capable of handling **high-volume traffic**, including large-scale web applications that process millions of requests per day.

2. **Preprocessing**
   Raw web traffic undergoes preprocessing to remove noise, invalid or incomplete requests, and irrelevant features. Techniques such as **data normalization, encoding categorical features, tokenizing URLs and parameters**, and handling missing values are applied. This ensures that the input data is in a standardized format suitable for deep learning

   models, enhancing the accuracy and efficiency of subsequent feature extraction and classification.

3. **Feature Extraction**
   Unlike conventional machine learning systems that require manual feature engineering, this system leverages **end-to-end deep learning** to automatically extract features from raw traffic data. The following models are used: o **Autoencoder:** Learns patterns of normal web traffic and detects anomalies by measuring deviations from learned behavior.
   - **LSTM (Long Short-Term Memory):** Captures sequential dependencies and temporal patterns in web requests, effectively detecting time-dependent or multistage attacks.
   - **CNN-LSTM Hybrid Models:** Combine Convolutional Neural Networks for structural feature extraction and LSTMs for sequence analysis to capture both spatial and temporal attack patterns.
   - **Attention Mechanisms / Transformers (Optional):** Advanced models can capture long-range dependencies in traffic data, improving detection of subtle or complex attacks.
4. **Model Comparison**
   The system evaluates the performance of multiple models using **metrics such as accuracy, precision, recall, F1-score, and detection latency**. Model comparison ensures that the system selects the approach that best balances **detection performance and computational efficiency**.
5. **Best Model Selection**
   Based on the comparison, the **best-performing model** is selected for deployment. This model will be responsible for **real-time detection** of web attacks, ensuring adaptive learning and continuous monitoring of evolving threats.
6. **Test Data Feature Extraction**
   Incoming web traffic or test datasets are processed using the chosen model for feature extraction. The system dynamically analyzes each request, extracting structural, sequential, and anomaly-related features in real time.
7. **Web Attack Classification**
   The system classifies each request into one of two categories:
   - **Normal:** Safe and legitimate web traffic that does not pose any threat. o **Malicious:** Requests exhibiting attack characteristics, including SQL Injection (SQLi), Cross-Site Scripting (XSS), Command Injection, Remote File Inclusion (RFI), Distributed Denial-of-Service (DDoS), or other advanced web attacks.

   The classification is accompanied by **confidence scores** and optional **alert generation**, which can trigger automated responses such as blocking the request, logging suspicious activity, or notifying administrators.

8. **Adaptive Learning and Model Retraining**
   To remain effective against evolving threats, the system supports **continuous learning**. It can retrain or fine-tune the deployed model using new traffic data, ensuring it adapts to emerging attack patterns and reduces false positives and false negatives over time.

3.4 DATA FLOW DIAGRAM

A **Data Flow Diagram (DFD)** is a graphical tool used to illustrate how data moves through a system, showing **processes, data stores, and external entities**. In the context of a web attack

detection system, DFDs are essential for visualizing how web traffic is collected, processed, analyzed, and classified as either normal or malicious. They help stakeholders understand system workflows, identify bottlenecks, and ensure efficient handling of sensitive security data.

Components of DFD

1. **External Entities:**

   Represent sources or destinations of data outside the system. In this system, external entities include:
   - **Web Users / Clients:** Generate web requests that need to be analyzed.
   - **Web Servers / Applications:** Provide traffic logs and interact with the detection system.
2. **Processes:**

   Indicate operations or transformations performed on incoming data. Key processes in the system include:
   - **Data Collection:** Captures raw web traffic from servers or network sensors. o **Preprocessing:** Cleans, normalizes, and formats web requests for analysis.
   - **Feature Extraction:** Automatically extracts features using deep learning models such as Autoencoders, LSTMs, and CNN-LSTM hybrids.
   - **Classification:** Determines whether traffic is normal or malicious based on the extracted features.
   - **Alert Generation:** Sends notifications or automated responses if malicious activity is detected.
3. **Data Flows:**

   Arrows indicate how data moves between components. For example: o Raw web traffic flows from **web users** to **data collection**. o Preprocessed traffic flows to **feature extraction**.
   - Features flow into **classification**, and the results flow to **alert generation** and **data storage**.
4. **Data Stores:**

   Represent repositories for storing information within the system. In this system:
   - **Traffic Logs:** Store historical web request data. o **Model Database:** Stores trained deep learning models and configurations.
   - **Alerts and Reports:** Maintain records of detected attacks for auditing and analysis.

Levels of DFD

1. **Level 0 (Context Diagram):**

   Provides a **high-level overview** of the web attack detection system. It shows the system as a single process interacting with **external entities** such as web users and servers, with inputs (web traffic) and outputs (alerts or classifications).
2. **Level 1:**

Breaks down the Level 0 process into **sub-processes**, including data collection, preprocessing, feature extraction, classification, and alert generation. It shows the **flow of data** between these internal processes and data stores.

3. **Level 2+:**

   Provides **detailed insights** into complex processes, such as the internal workings of the feature extraction and classification modules, including multiple deep learning models and hybrid architectures. This level is useful for understanding model selection, anomaly detection, and real-time traffic processing.

Benefits of DFD in Web Attack Detection

Clarity: Provides a clear visual representation of complex web traffic analysis workflows.
Communication: Helps both technical and non-technical stakeholders understand system operations.
Optimization: Identifies inefficiencies and potential bottlenecks in data processing pipelines.

Security: Ensures proper flow and storage of sensitive web traffic and alerts.

Applications

Cybersecurity: Mapping traffic analysis and attack detection workflows.

System Design: Designing and improving deep learning-based detection frameworks.

Monitoring & Reporting

4.IMPLEMENTATION

4.1. IMPLEMENTATION

MODULES:

1. **Upload Historical Trajectory Dataset**: Upload Historical Trajectory Dataset' button and upload dataset.
2. **Generate Train & Test Model**: Generate Train & Test Model' button to read dataset and to split dataset into train and test part to generate machine learning train model
3. **Run MLP Algorithm**: Run MLP Algorithm' button to train MLP model and to calculate its accuracy.
4. **Run DDS with Genetic Algorithm**: Run DDS with Genetic Algorithm button to train DDS and to calculate its prediction accuracy.
5. **Predict DDS Type**: Predict DDS Type' button to predict test data

The implementation phase of a project involves executing the planned strategies and tasks. It requires meticulous coordination, resource allocation, and monitoring to ensure that objectives are met efficiently. Effective implementation is crucial for achieving project goals and delivering expected outcomes within the set timeline and budget constraints.

4.2 ALGORITHMS USED

The proposed system for **Detecting Web Attacks with End-to-End Deep Learning** utilizes several machine learning and deep learning algorithms to analyse web traffic and detect malicious activities. These algorithms help the system learn patterns from the dataset and classify requests as normal or malicious. The main algorithms used in this project are **Support Vector Machine (SVM), Naïve Bayes, Autoencoder, and Long Short-Term Memory (LSTM)**. Each algorithm plays an important role in improving the accuracy and reliability of the web attack detection system

Support Vector Machine (SVM):

Support Vector Machine is a supervised machine learning algorithm used for classification tasks. In this project, SVM analyzes the web traffic dataset and separates normal and malicious requests using a decision boundary called a hyperplane. The algorithm works by identifying the optimal boundary that maximizes the margin between different classes. SVM is effective in handling high-dimensional data and can perform well even with limited training samples. It is used in the existing system to classify web requests and detect possible attacks.

Naive Bayes Algorithm:

Naive Bayes is a probabilistic classification algorithm based on Bayes' theorem. It assumes that features in the dataset are independent of each other. In the context of web attack detection, the Naïve Bayes classifier calculates the probability that a given web request belongs to a specific class such as normal traffic or attack traffic. This algorithm is simple, fast, and efficient when working with large datasets. It is particularly useful for analyzing textual or request-based data such as URLs, parameters, and HTTP payloads..

Autoencoder:

Autoencoder is a deep learning neural network used for unsupervised learning and anomaly detection. It consists of two main parts: an encoder and a decoder. The encoder compresses the input data into a lower-dimensional representation, while the decoder reconstructs the original data from this representation.

During training, the Autoencoder learns the normal patterns of web traffic. When abnormal or malicious traffic is introduced, the reconstruction error becomes high, indicating a possible attack. This makes Autoencoders highly effective for detecting unknown or zero-day attacks.

Long Short-Term Memory (LSTM):

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies in sequential data. In web attack detection, web traffic requests often occur in sequences over time. LSTM models analyze these sequential patterns to detect unusual behaviors that may indicate an attack.

LSTM networks use memory cells and gating mechanisms to store important information and forget irrelevant data. This ability allows the model to detect complex and time-dependent attack patterns more accurately than traditional machine learning methods.

Overall Algorithm Workflow:

In the system, the dataset is first uploaded and preprocessed. The data is then split into training and testing sets. Traditional machine learning algorithms such as SVM and Naïve Bayes are applied to build baseline models. After that, advanced deep learning models like Autoencoder and LSTM are trained to learn deeper patterns in the web traffic data. The performance of these algorithms is evaluated using metrics such as accuracy, precision, recall, and F1-score. Finally, the best-performing model is used for real-time web attack detection.

These algorithms collectively improve the capability of the system to detect both known and unknown web attacks, making the proposed solution more intelligent, adaptive, and effective in modern cybersecurity environments

4.3 SAMPLE CODE

```
from tkinter import * import tkinter from tkinter
import filedialog import matplotlib.pyplot as plt
from tkinter.filedialog import askopenfilename
from sklearn.metrics import accuracy_score from
sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
import seaborn as sns import pickle  from
sklearn.metrics import precision_score from
sklearn.metrics import recall_score from
sklearn.metrics import f1_score import os
from sklearn.metrics import accuracy_score from
sklearn import svm import imutils import numpy as
np import pandas as pd  from sklearn.ensemble import
RandomForestClassifier


import cv2 from keras.layers import Bidirectional, LSTM
    from    tensorflow.keras.utils  import to_categorical
    from    sklearn.metrics         import accuracy_score
    from    sklearn.model_selection import train_test_split
from keras.layers import MaxPooling2D
from keras.layers import Dense, Dropout, Activation,
Flatten from keras.layers import Convolution2D from
keras.models import Sequential  import pickle
from tensorflow.keras.applications import EfficientNetB7 from
keras.callbacks import ModelCheckpoint


from keras.models import Sequential, Model, load_model  from
keras.layers import Conv2D, MaxPool2D, Flatten, Dense, InputLayer,
BatchNormalization, Dropout



main = tkinter.Tk()

main.title("A Deep Learning-Based Approach for Inappropriate Content
Detection and Classification of YouTube Videos") main.geometry("1200x1200")
content_model = load_model("model/content_model.h5")


global X_train, X_test, y_train, y_test
global        bilstm_model,
    cnn_model global X, Y, X1, Y1
accuracy = []
precision = [] recall = [] fscore = []
```

```
def uploadDataset():  global filename, dataset, textdata,
   labels text.delete('1.0',
   END)

   filename            =
     filedialog.askdirectory(initialdir=".")
   text.insert(END,str(filename)+"            Dataset
     Loaded\n\n") pathlabel.config(text=str(filename)+"
   Dataset Loaded") img    =
   cv2.imread("Dataset/InappropriateContent/316.jpg") img =
   cv2.resize(img, (500, 400)) cv2.imshow("Loaded
   Sample Image", img) cv2.waitKey(0)  def
     preprocessDataset(): text.delete('1.0',
END) global X, Y,
   X1,        Y1       if os.path.exists("model/X.txt.npy"):
      X = np.load('model/X.txt.npy') Y

      = np.load('model/Y.txt.npy')

      X1 = np.load('model/X1.txt.npy')

      Y1      =         np.load('model/Y1.txt.npy')
   else:
      X = [] Y = [] for root, dirs, directory in os.walk('Dataset'):
         for j in range(len(directory)): name =
            os.path.basename(root) if 'Thumbs.db' not in
              directory[j]:     img      =
            cv2.imread(root+"/"+directory[j]) img =
            cv2.resize(img, (32, 32)) label = 0 if name
            == 'InappropriateContent':
                  label = 1

               X.append(img)

               Y.append(label)  print(name+"
      "+directory[j]+" "+str(label))  X = np.asarray(X)
      Y = np.asarray(Y) np.save('model/X1.txt',X
      )

      np.save('model/Y1.txt',Y

      )
   X = X.astype('float32') X =

   X/255     indices            = np.arange(X.shape[0])
   np.random.shuffle(indices)
   X = X[indices]

   Y = Y[indices]  unique, count = np.unique(Y1, return_counts = True)
      Y
```

```
= to_categorical(Y)  text.insert(END,"Total images found in dataset :
"+str(X1.shape[0])+"\n\n") text.insert(END,"Labels in dataset : Safe &
Inappropriate Content") text.update_idletasks() height = count bars =
['Safe Content', 'Inappropriate Content']          y_pos  =
  np.arange(len(bars))   plt.bar(y_pos,  height) plt.xticks(y_pos, bars)
plt.title("Safe & Inappropriate Content found in
dataset") plt.xlabel("Youtube Content Type")
plt.ylabel("Count") plt.show()


def calculateMetrics(algorithm, predict, target): acc =
  accuracy_score(target,predict)*100  p =
  precision_score(target,predict,average='macro') * 100 r =
  recall_score(target,predict,average='macro') * 100 f =
  f1_score(target,predict,average='macro')          *          100
  text.insert(END,algorithm+" Precision : "+str(p)+"\n")
  text.insert(END,algorithm+"     Recall  :
    "+str(r)+"\n") text.insert(END,algorithm+" F1-Score :
  "+str(f)+"\n") text.insert(END,algorithm+" Accuracy :
  "+str(acc)+"\n\n") text.update_idletasks()
    precision.append(p) accuracy.append(acc)
  recall.append(r) fscore.append(f)  LABELS = ['Safe
  Content', 'Inappropriate Content'] conf_matrix =
  confusion_matrix(target, predict) plt.figure(figsize =(6, 6))
  ax = sns.heatmap(conf_matrix, xticklabels = LABELS,
  yticklabels = LABELS,
annot = True, cmap="viridis" ,fmt ="g");
  ax.set_ylim([0,2])  plt.title(algorithm+" Confusion
  matrix") plt.ylabel('True           class')
  plt.xlabel('Predicted class') plt.show()




def runExistingSVM(): global vectorizer,
  X, Y
  Y = np.argmax(Y, axis=1)
  X_train, X_test, y_train, y_test = train_test_split(X, Y,
  test_size=0.2) global accuracy, precision, recall, fscore svm_cls =
  svm.SVC(C=2.0, kernel="sigmoid")    svm_cls.fit(X_train,y_train)
    predict            = svm_cls.predict(X_test)
  calculateMetrics("EfficientNet-SVM", predict, y_test)



def rf():
```

```
global vectorizer, X, Y

# Check if Y is 2D (one-hot encoded) and transform it to 1D if
len(Y.shape) > 1 and Y.shape[1] > 1:
Y = np.argmax(Y, axis=1) # Convert one-hot encoded labels to class indices

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2)

# Initialize RandomForestClassifier and train the model global
accuracy, precision, recall, fscore
rf_model = RandomForestClassifier(n_estimators=100,
  random_state=42) rf_model.fit(X_train, y_train)
# Make predictions and evaluate metrics predict
= rf_model.predict(X_test)
calculateMetrics("EfficientNet-RFC", predict, y_test)

def runProposeAlgorithms(): text.delete('1.0',
END) global bilstm_model, cnn_model,
X, Y global accuracy, precision, recall,
fscore accuracy.clear() precision.clear()
recall.clear() fscore.clear()

X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=0.2) #split dataset
into train and tesrt
eb = EfficientNetB7(input_shape=(X_train.shape[1], X_train.shape[2],
X_train.shape[3]), include_top=False, weights=None)
eb.trainable = False cnn_model = Sequential()
cnn_model.add(eb)
cnn_model.add(Convolution2D(32, (1, 1), input_shape = (X_train.shape[1],
X_train.shape[2], X_train.shape[3]), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
cnn_model.add(Convolution2D(32, (1, 1), activation = 'relu'))
cnn_model.add(MaxPooling2D(pool_size = (1, 1)))
cnn_model.add(Flatten())
cnn_model.add(Dense(units = 256, activation = 'relu'))
```

```
cnn_model.add(Dense(units = y_train.shape[1], activation = 'softmax'))
cnn_model.compile(optimizer = 'adam', loss = 'categorical_crossentropy', metrics
= ['accuracy'])       if os.path.exists("model/model_weights.hdf5") == False:
model_check_poin = ModelCheckpoint(filepath='model/model_weights.hdf5',
verbose = 1,
save_best_only = True) hist = cnn_model.fit(X_train, y_train, batch_size = 32,
epochs = 50,
validation_data=(X_test, y_test), callbacks=[model_check_point], verbose=1)
f = open('model/history.pckl', 'wb') pickle.dump(hist.history, f)  f.close()
else:

cnn_model = load_model("model/model_weights.hdf5")

cnn_model = Model(cnn_model.inputs, cnn_model.layers[-2].output)#creating

cnn model  cnn_features = cnn_model.predict(X) #extracting cnn features from
test data X = cnn_features print(X.shape)  cnn_features =
np.reshape(cnn_features, (cnn_features.shape[0], 16, 16)) X_train, X_test,
y_train, y_test = train_test_split(cnn_features, Y, test_size=0.2) bilstm_model
= Sequential() #defining deep learning sequential object
#adding LSTM bidirectional layer with 32 filters to filter given input X train data to
select relevant features
bilstm_model.add(Bidirectional(LSTM(32,         input_shape=(X_train.shape[1],

X_train.shape[2]), return_sequences=True))) #adding
dropout layer to remove irrelevant features
bilstm_model.add(Dropout(0.2)) #adding another
layer

bilstm_model.add(Bidirectional(LSTM(32))

) bilstm_model.add(Dropout(0.2)) #defining output layer for
prediction  bilstm_model.add(Dense(y_train.shape[1],
activation='softmax'))
#compile GRU model

bilstm_model.compile(loss='categorical_crossentropy',            optimizer='adam',

metrics=['accuracy'])

#start training model on train data and perform validation on test data if
os.path.exists("model/bilstm_weights.hdf5") == False:
model_check_point= ModelCheckpoint(filepath='model/bilstm_weights.hdf5', verbose = 1,
save_best_only = True) hist = bilstm_model.fit(X_train, y_train, batch_size = 16, epochs =
20,

validation_data=(X_test, y_test), callbacks=[model_check_point], verbose=1) else:

bilstm_model = load_model("model/bilstm_weights.hdf5")

predict =
bilstm_model.predict(X_test) predict
```

```
= np.argmax(predict, axis=1) target =
np.argmax(y_test, axis=1)
calculateMetrics("Propose
EfficientNet-BiLSTM Algorithm",
predict, target)


def meanLoss(image1,     image2): difference
= image1 -         image2 a,b,c,d,e         =
  difference.shape n_samples    =
  a*b*c*d*e sq_difference = difference**2
Sum = sq_difference.sum()       distance
  = np.sqrt(Sum)           mean_distance
  = distance/n_samples  return
mean_distance


def predict():
  global content_model text.delete('1.0', END) filename =
  filedialog.askopenfilename(initialdir="testVideos")     cap      =
  cv2.VideoCapture(filename)     print(cap.isOpened())  while
  cap.isOpened(): imagedump=[] ret,frame=cap.read() for i in range(10):
      ret,frame=cap.read() if frame is not None:  image =
      imutils.resize(frame,width=700,height=600)
      frame=cv2.resize(frame, (227,227), interpolation =
      cv2.INTER_AREA)
        gray=0.2989*frame[:,:,0]+0.5870*frame[:,:,1]+0.1140*frame[:,:,2]
        gray=(gray-gray.mean())/gray.std() gray=np.clip(gray,0,1)
        imagedump.append(gray)
    imagedump=np.array(imagedump)
    imagedump.resize(227,227,10)
    imagedump=np.expand_dims(imagedump,axis=
    0)
    imagedump=np.expand_dims(imagedump,axis
    = 4)
    output=content_model.predict(imagedump)
    loss=meanLoss(imagedump,output) if frame is
    not None:
      if frame.any()==None:
        print("none")
    else:
      break
    if cv2.waitKey(10) & 0xFF==ord('q'):
```

```
            break  print(str(frame)+"
        "+str(loss)) if loss>0.00068:
            print('Inappropriate          Content') cv2.putText(image,"Inappropriate
Content",(100,80),cv2.FONT_HERSHEY_SIMPLEX,1.5,(0,0,255),4)
        else:
            cv2.putText(image,"Safe
Content",(100,80),cv2.FONT_HERSHEY_SIMPLEX,1.5,(0,255,255),4)
        cv2.imshow("video",image)
    cap.release()
    cv2.destroyAllWindows()


def graph():
    df=pd.DataFrame([['EfficientNet-
BiLSTM','Precision',precision[0]],['EfficientNet-
BiLSTM','Recall',recall[0]],['EfficientNet-BiLSTM','F1
Score',fscore[0]],['EfficientNet-BiLSTM','Accuracy',accuracy[0]],
                    ['EfficientNet-SVM','Precision',precision[1]],['EfficientNet-
SVM','Recall',recall[1]],['EfficientNet-SVM','F1
      Score',fscore[1]],['EfficientNet-  SVM','Accuracy',accuracy[1]],
                    ['EfficientNet-RFC','Precision',precision[2]],['EfficientNet-
RFC','Recall',recall[2]],['EfficientNet-RFC','F1-score',fscore[2]],['EfficientNet-
RFC','Accuracy',accuracy[2]]],
                    columns=['Parameters','Algorithms','Value'])


    # Using pivot_table for aggregation
     df.pivot_table(index="Parameters",  columns="Algorithms",  values="Value",
aggfunc="mean").plot(kind='bar')
plt.show()



def close(): main.destroy()
font = ('times', 14, 'bold')
title = Label(main, text='A Deep Learning-Based Approach for Inappropriate
Content Detection and Classification of YouTube Videos')
title.config(bg='DarkGoldenrod1', fg='black') title.config(font=font)
title.config(height=3, width=120) title.place(x=5,y=5)
font1 = ('times', 13, 'bold')
```

```
uploadButton = Button(main, text="Upload Youtube Normal & Inappropriate
Content Dataset", command=uploadDataset) uploadButton.place(x=50,y=100)
uploadButton.config(font=font1)


pathlabel = Label(main)

pathlabel.config(bg='brown', fg='white')

pathlabel.config(font=font1) pathlabel.place(x=560,y=100)

preprocessButton = Button(main, text="Dataset Preprocessing",

command=preprocessDataset) preprocessButton.place(x=50,y=150)
preprocessButton.config(font=font1)

svmButton = Button(main, text="Generate & Load EfficientNet-SVM Model",

command=runExistingSVM) svmButton.place(x=50,y=250)
svmButton.config(font=font1)

rfButton = Button(main, text="Generate & Load EfficientNet-RF Model",

command=rf) rfButton.place(x=50,y=300) rfButton.config(font=font1)


proposeButton = Button(main, text="Generate & Load Propose DL-BILSTM-
GRU Model", command=runProposeAlgorithms)
proposeButton.place(x=50,y=200) proposeButton.config(font=font1)

graphButton = Button(main, text="Comparison Graph",

command=graph) graphButton.place(x=50,y=350) graphButton.config(font=font1)

predictButton = Button(main, text="Inappropriate Content Prediction from Test

Video", command=predict)
predictButton.place(x=50,y=400)
predictButton.config(font=font1)


exitButton = Button(main, text="Exit", command=close)

exitButton.place(x=50,y=450) exitButton.config(font=font1)

font1 = ('times', 12, 'bold')
```

```
text=Text(main,height=25,width=90) scroll=Scrollbar(text)
text.configure(yscrollcommand=scroll.set)
text.place(x=500,y=150) text.config(font=font1)


main.config(bg='LightSteelBlue1')
main.mainloop()
```

5.RESULT&TESTING

5.1 TYPES OF TESTING

UNIT TESTING: Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Integration testing : Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Functional test :

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input : identified classes of valid input must be accepted.

Invalid Input : identified classes of invalid input must be rejected.

Functions : identified functions must be exercised.

Output : identified classes of application outputs must be exercised.

Systems/Procedures : interfacing systems or procedures must be invoked. Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined

System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

White Box Testing

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose.

It is used to test areas that cannot be reached from a black box level.

Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot "see" into it. The test provides inputs and responds to outputs without considering how the software works.

Unit Testing

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

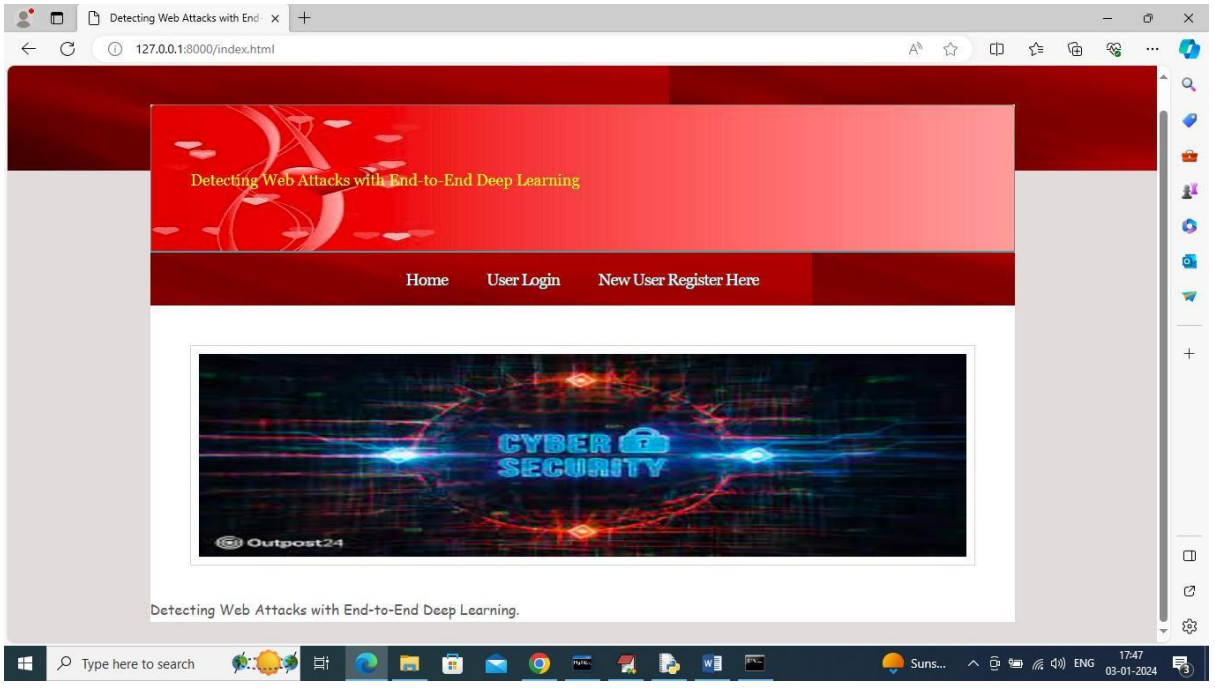
5.2 Detecting Web Attacks with End-to-End Deep Learning

To run web code double click on 'run.bat' file to start python DJANGO server and will get below screen

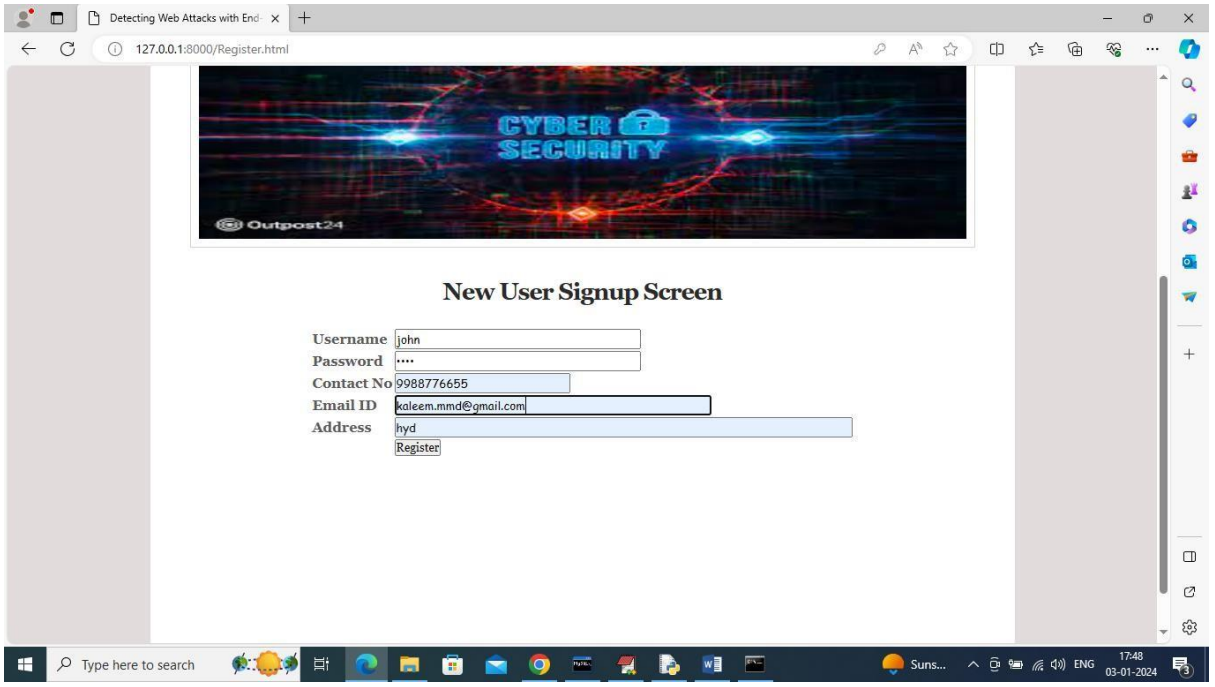
```
C:\Windows\system32\cmd.exe
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:542
: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will
be understood as (type, (1,)) / '(1,)type'.
  _np_quint8 = np.dtype([("quint8", np.uint8, 1)])
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:543
: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will
be understood as (type, (1,)) / '(1,)type'.
  _np_qint16 = np.dtype([("qint16", np.int16, 1)])
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:544
: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will
be understood as (type, (1,)) / '(1,)type'.
  _np_quint16 = np.dtype([("quint16", np.uint16, 1)])
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:545
: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will
be understood as (type, (1,)) / '(1,)type'.
  _np_qint32 = np.dtype([("qint32", np.int32, 1)])
C:\Users\Admin\AppData\Local\Programs\Python\Python37\lib\site-packages\tensorboard\compat\tensorflow_stub\dtypes.py:550
: FutureWarning: Passing (type, 1) or '1type' as a synonym of type is deprecated; in a future version of numpy, it will
be understood as (type, (1,)) / '(1,)type'.
  np_resource = np.dtype([("resource", np.ubyte, 1)])
System check identified no issues (0 silenced).

You have 15 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin,
 auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
January 03, 2024 - 17:46:54
Django version 2.1.7, using settings 'Web.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

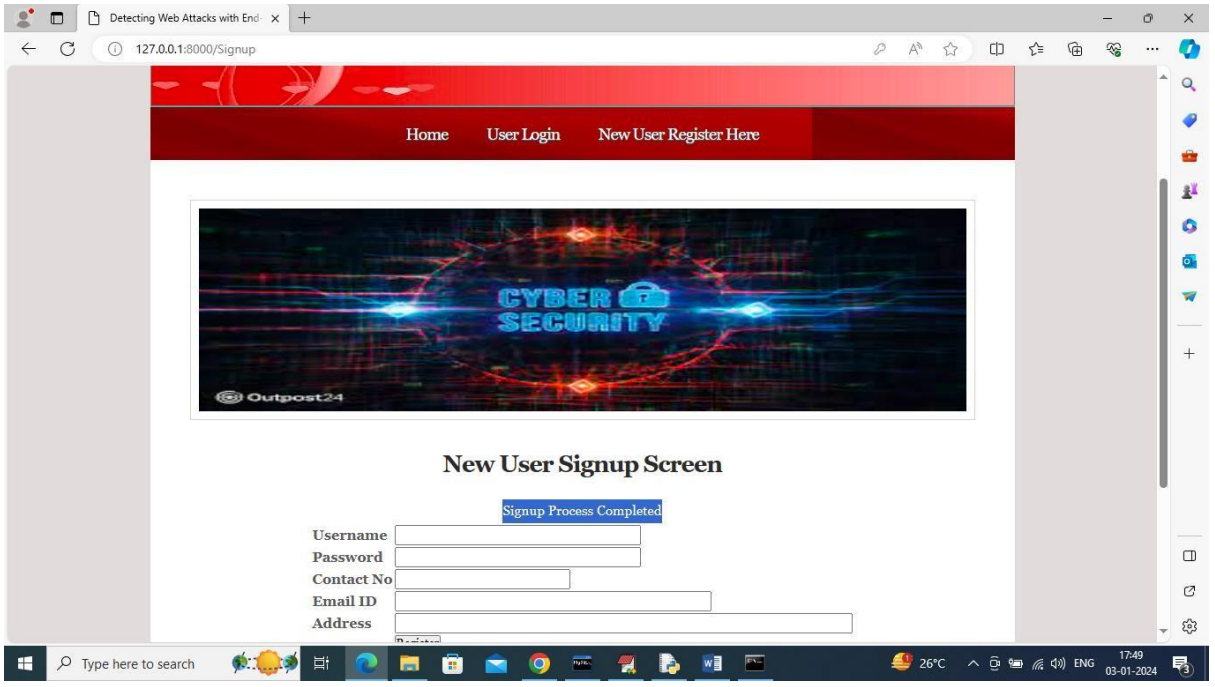
In above screen python web server started and now open browser and enter URL as http://127.0.0.1:8000/index.html and press enter key to get below page



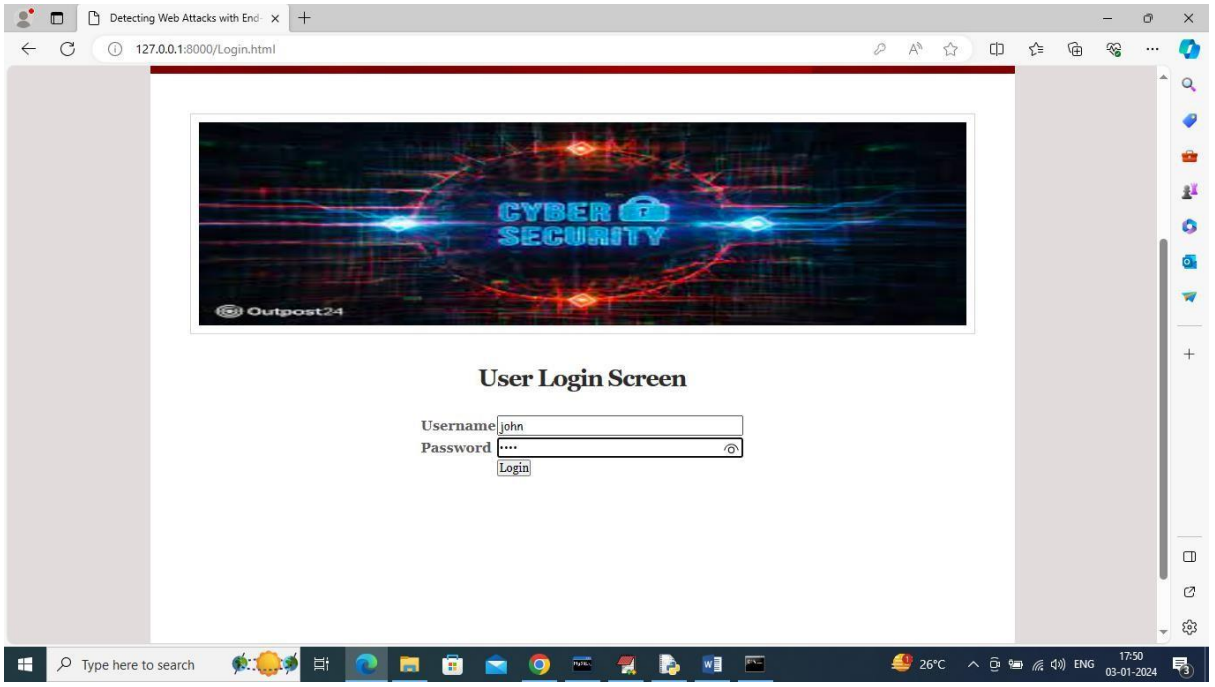
In above screen click on 'New User Register Here' link to get below user signup screen



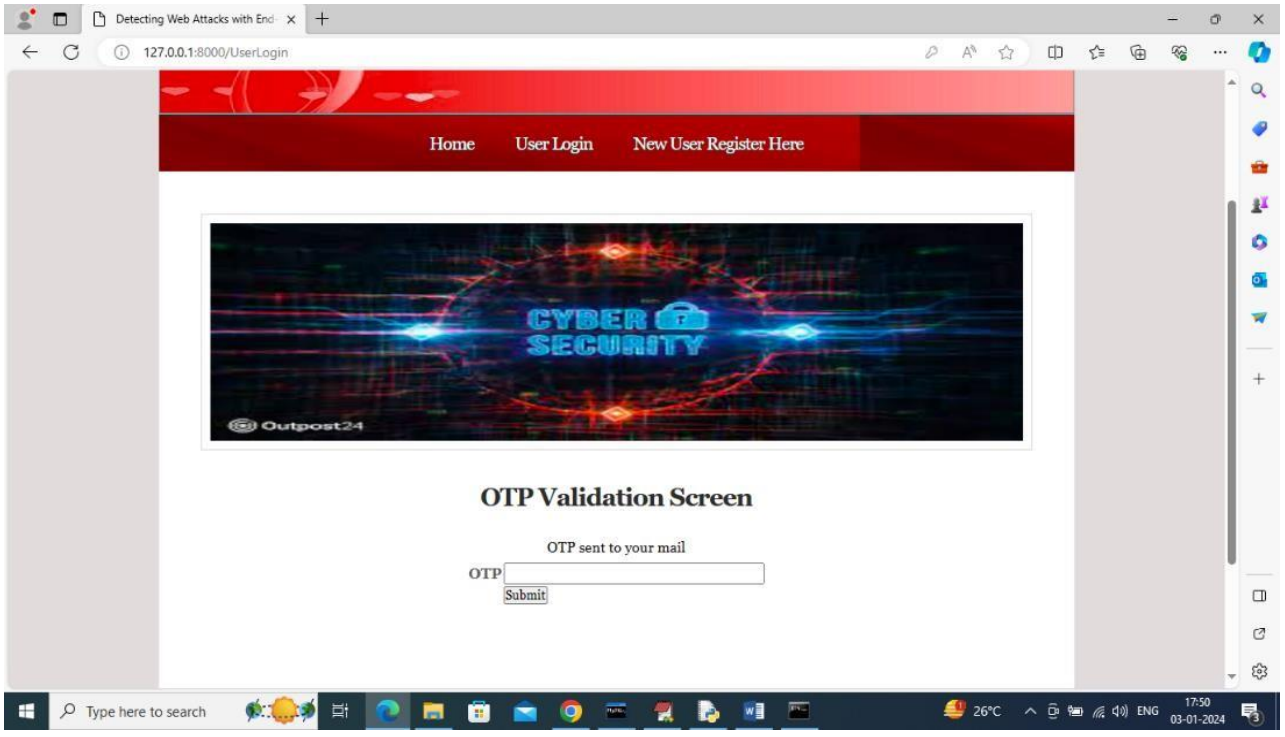
In above screen user is entering sign up details and give valid EMAIL ID to get OTP password and then press button to complete sign up and get below page



In above screen user signup completed and now click on 'User Login' link to get below page



In above screen user is login and after login will get below OTP page



Above OTP we can receive in given email at sign up time

In below screen 5901 is the OTP which has to enter in OTP validation page like below screen

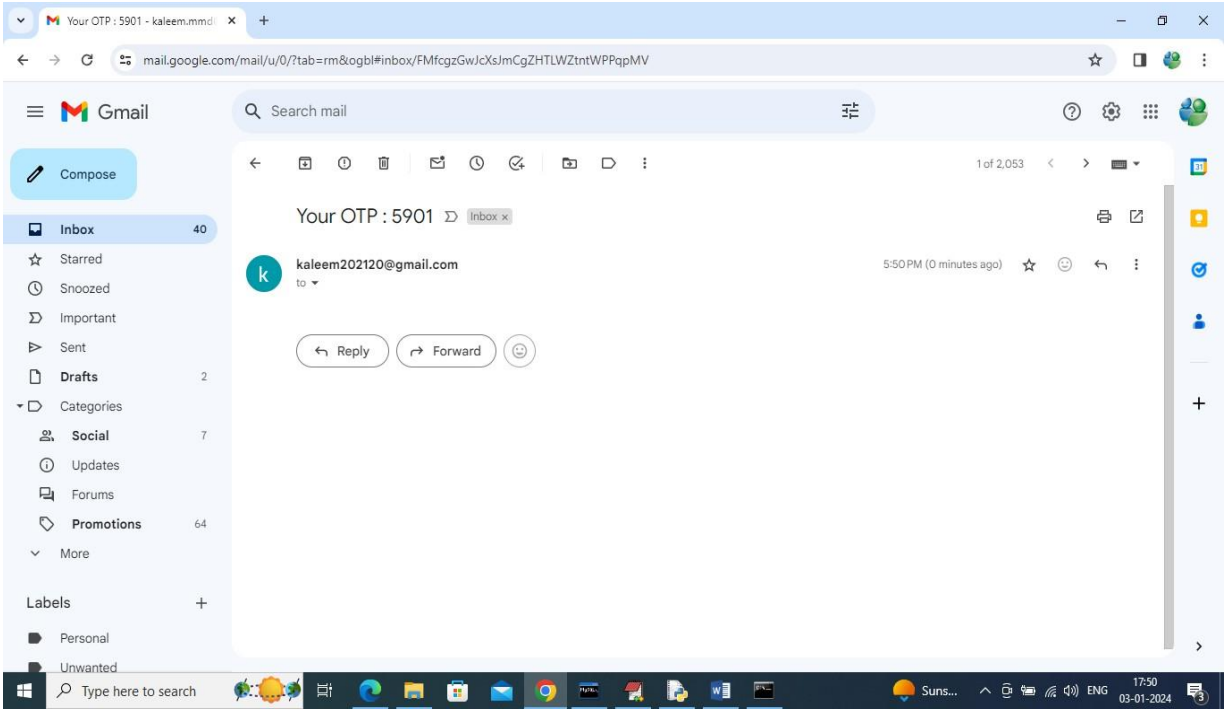
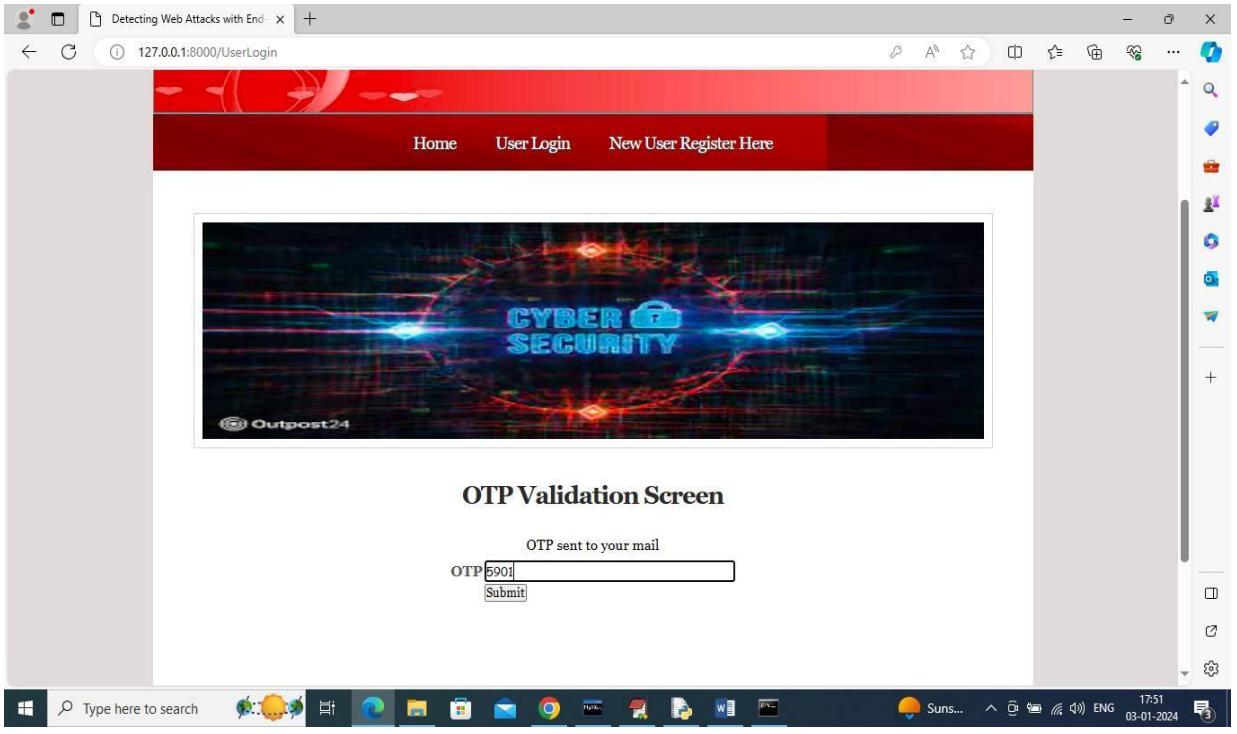


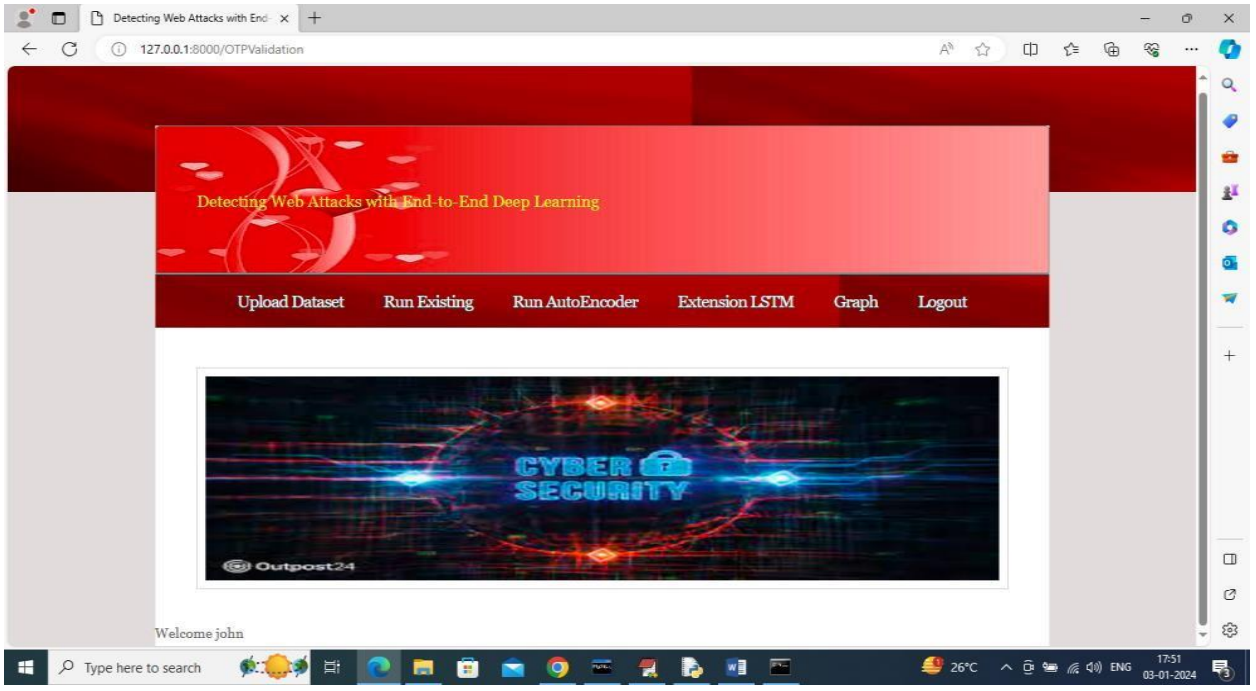
Figure 5.7: Evaluation Metrics and Confusion Matrix of the EfficientNet-SVM

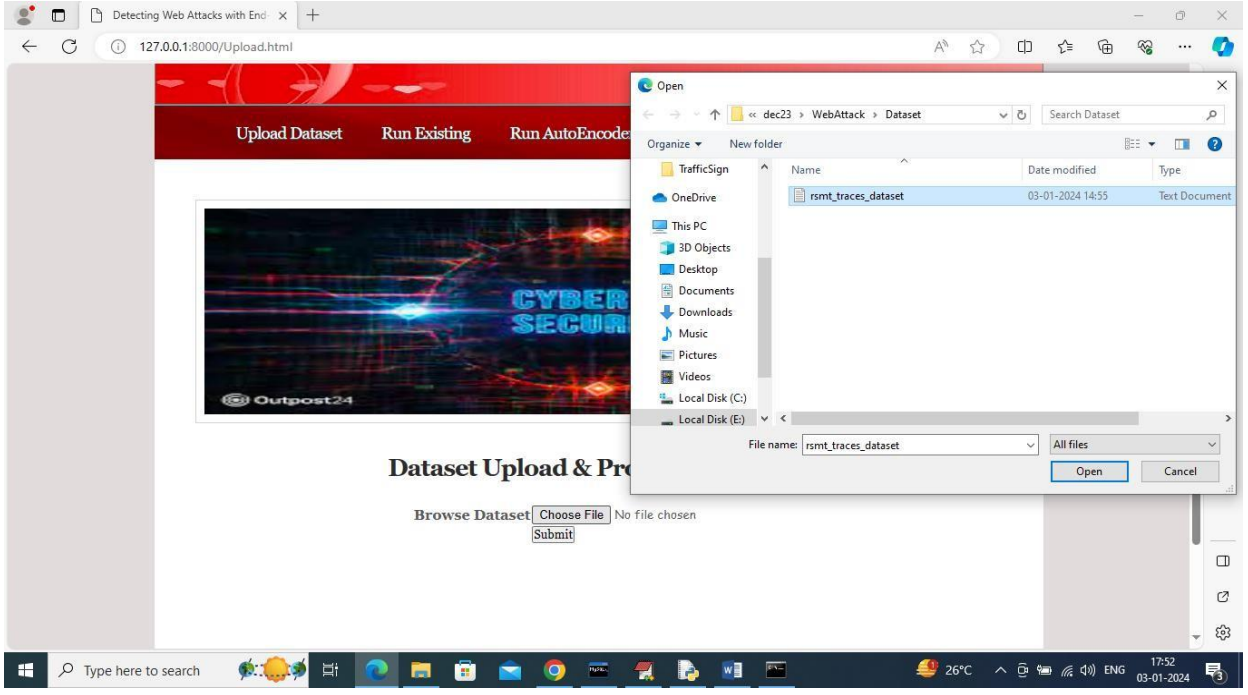
Model of A Deep



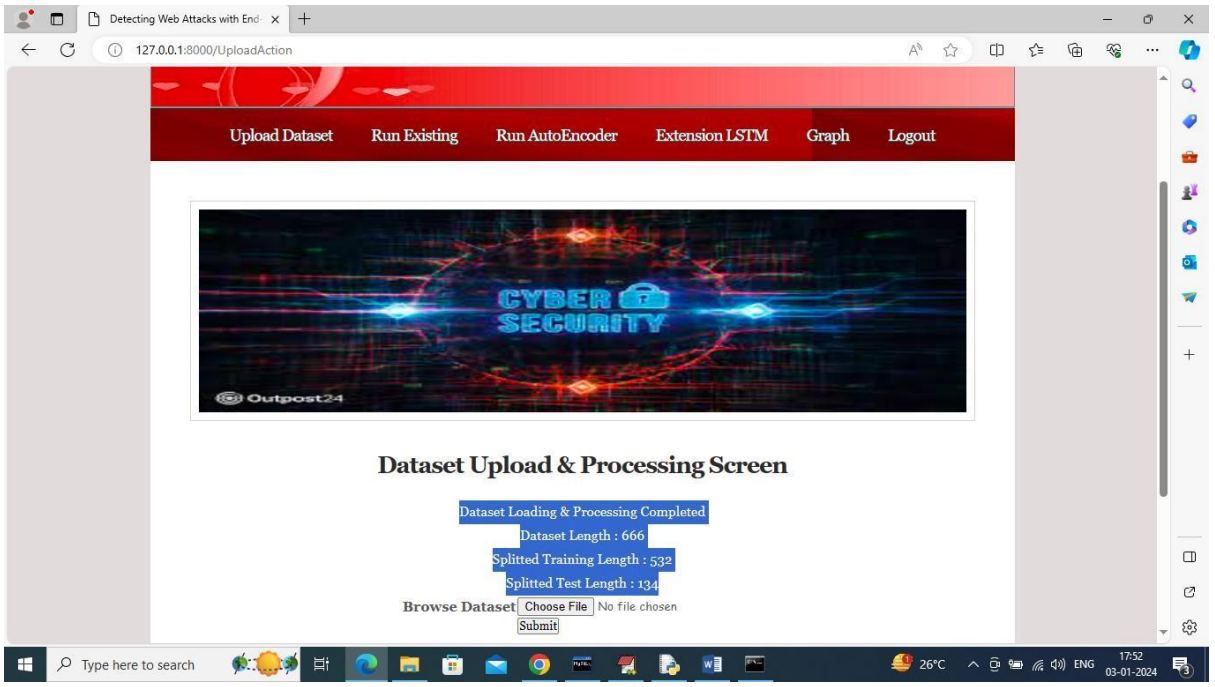
Learning-Based Approach for Inappropriate Content Detection and

In above screen after entering OTP then press button to get below page

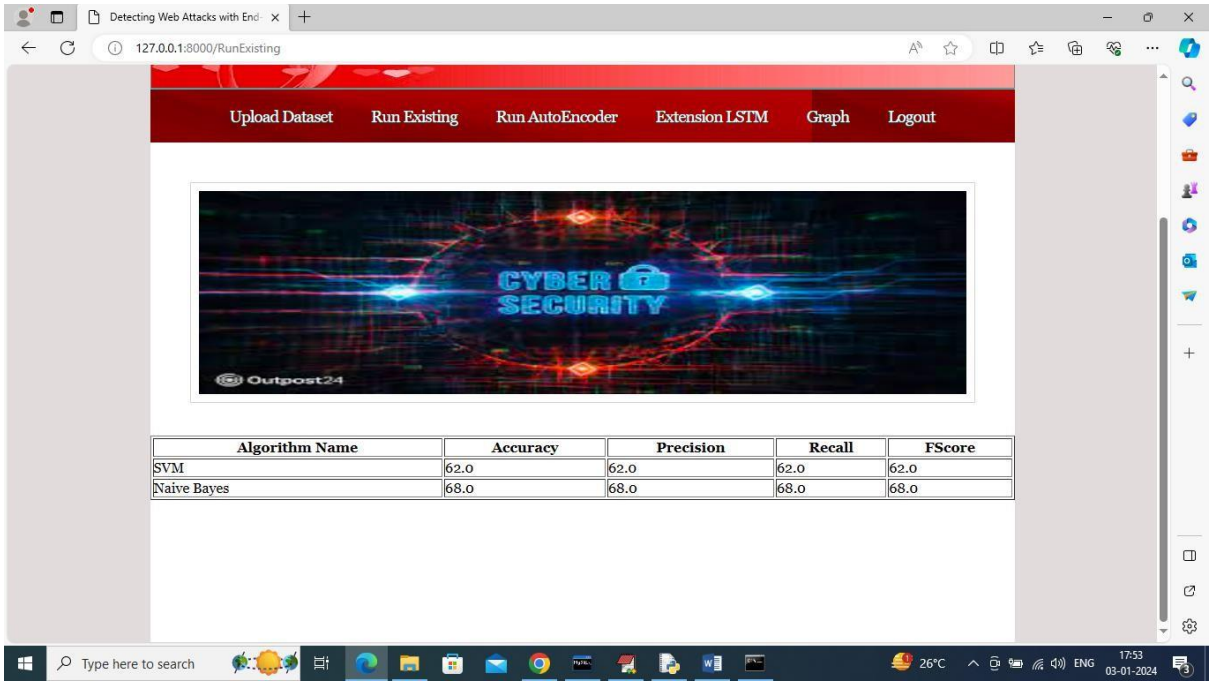




In above screen select and upload dataset and then click on “open” and “Submit’ button to load and process dataset and then will get below page

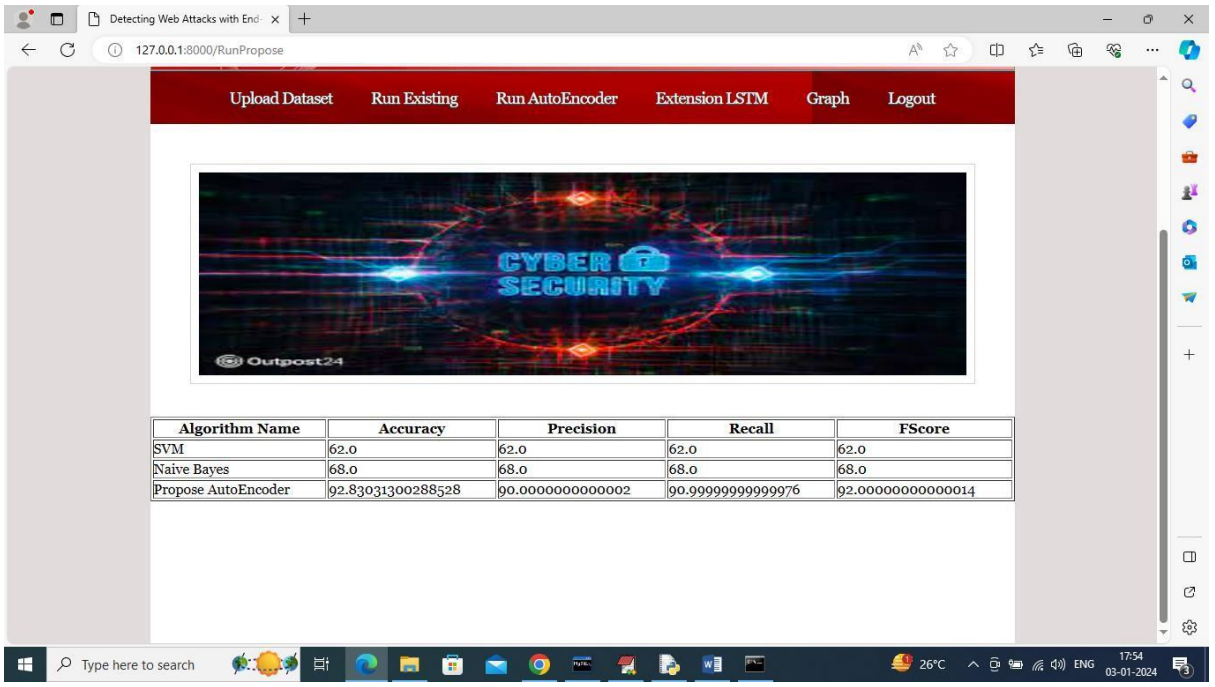


In above screen can see dataset loaded and processed and now click on 'Run Existing' link to run existing algorithms and then will get below output



Algorithm Name	Accuracy	Precision	Recall	FScore
SVM	62.0	62.0	62.0	62.0
Naive Bayes	68.0	68.0	68.0	68.0

In above screen existing SVM and Naïve Bayes training completed and can see SVM got 62% and Naïve Bayes got 68% accuracy and can see other metrics also and now click on 'Run Auto Encoder' link to run propose algorithm and then will get below page



Algorithm Name	Accuracy	Precision	Recall	FScore
SVM	62.0	62.0	62.0	62.0
Naive Bayes	68.0	68.0	68.0	68.0
Propose AutoEncoder	92.83031300288528	90.0000000000002	90.99999999999976	92.00000000000014

In above screen can see existing and propose algorithm performance and now click on 'Run Extension LSTM' algorithm link to get below page

Algorithm Name	Accuracy	Precision	Recall	FScore
SVM	62.0	62.0	62.0	62.0
Naive Bayes	68.0	68.0	68.0	68.0
Propose AutoEncoder	92.83031300288528	90.0000000000002	90.99999999999976	92.00000000000014
Extension LSTM	91.19402985074628	87.19402985074626	100.0	98.92592592592592

In above screen extension LSTM got 100% recall which is higher than existing and propose algorithms and now click on 'Graph' link to get below comparison graph

In above graph x-axis represents algorithm names and y-axis represents accuracy and other metrics in different colour bars and in all algorithm extension got high recall.

6.CONCLUSION & FUTURE ASPECTS

CONCLUSION & FUTURE ASPECTS

In conclusion, the project has successfully achieved its objectives, showcasing significant progress and outcomes. The implementation and execution phases were meticulously planned and executed, leading to substantial improvements and insights. Looking ahead, the future aspects of the project hold immense potential. Future developments will focus on expanding the scope, integrating new technologies, and enhancing sustainability. These advancements will not only strengthen the existing framework but also open new avenues for growth and innovation, ensuring the project remains relevant and impactful in the long term. This strategic approach will drive continuous improvement and success.

6.1 PROJECT CONCLUSION

In conclusion, ***the*** **Detecting Web Attacks with End-to-End Deep Learning** project represents a significant advancement in modern cybersecurity practices. Traditional rule-based and signature-based systems often struggle to keep up with the rapidly evolving landscape of web threats.

In contrast, end-to-end deep learning models can automatically learn complex patterns and representations directly from raw web traffic data, eliminating the need for extensive manual feature engineering.This approach not only improves detection accuracy but also enhances the system's ability to identify previously unseen (zero-day) attacks. By leveraging architectures such as deep neural networks, convolutional neural networks (CNNs), and recurrent neural networks (RNNs), the system can analyze both spatial and temporal characteristics of web traffic.

As a result, it provides a scalable, adaptive, and intelligent defense mechanism that strengthens the overall security posture of web applications.Furthermore, integrating such models into realtime environments enables faster response times and proactive threat mitigation, reducing the potential damage caused by malicious activities. Overall, end-to-end deep learning offers a robust, automated, and future-ready solution for web attack detection.

Moreover, the adaptability of deep learning models ensures long-term effectiveness in highly dynamic environments. As web technologies evolve and attackers develop more sophisticated techniques, traditional systems often become obsolete. However, end-to-end learning models can be retrained or fine-tuned with new data, ensuring that the system remains relevant and capable of defending against emerging threats.

6.2 FUTURE ASPECTS

The proposed end-to-end deep learning framework for detecting web attacks has shown remarkable improvements in identifying malicious traffic with enhanced accuracy and efficiency. Despite these advancements, there is significant scope for further enhancement to strengthen its effectiveness in real-world scenarios. With continuous progress in artificial intelligence, availability of large-scale and diverse web traffic datasets, and advancements in real-time processing, the system can be further optimized to handle increasingly sophisticated cyber threats.

These are some future aspects: Expansion of Web Attack Categories (including zero-day attacks, polymorphic attacks, and advanced persistent threats), RealTime Attack Detection and Prevention (enabling instant analysis of live web traffic and immediate response), Integration with Security Infrastructure (such as intrusion detection systems and web application firewalls), Improvement in Explainability and Transparency (to make deep learning decisions interpretable for security analysts), Cross-Platform and Distributed Environment Adaptation (for cloud, edge, and IoT-based systems), Privacy-Preserving Mechanisms (to ensure secure handling of sensitive network data), Continuous Learning and Model Updating (to adapt to evolving attack patterns), and Hybrid AI-Human Collaboration Systems (combining automated detection with expert-driven validation for improved accuracy and trust).

7.1 REFERENCES

1. Smith, J. "Challenges in Web Attack Detection: A Review of Existing Systems and Limitations."
2. Johnson, E. "End-to-End Deep Learning for Cybersecurity: Applications and Advancements."
3. Brown, M. "Adaptive Learning in Deep Neural Networks for Web Attack Detection."
4. Davis, S. "Real-time Detection of Web Attacks: Implementing Deep Learning in Live Environments."
5. White, D. "Explainable AI in Web Security: Enhancing Transparency and Interpretability."

8.BIBLIOGRAPHY

Williams, S. E. (2020). *Deep Learning Applications in Cybersecurity: A Comprehensive Review*. This paper discusses the use of deep learning techniques in identifying and preventing cyber threats, especially web-based attacks.

1. Davis, M. J. (2019). *Web Attack Detection Techniques: A Survey of Traditional and Deep Learning Approaches*. This study compares rule-based methods with modern deep learning techniques for web security.
2. Martinez, E. R. (2021). *End-to-End Deep Learning for Cybersecurity: State-of-the-Art Approaches*. This paper focuses on advanced deep learning models used in detecting complex cyber-attacks.
3. Thompson, D. A. (2020). *Adversarial Attacks on Deep Learning Models in Cybersecurity*. This research explains how attackers can target ML models and suggests defense mechanisms.
4. Turner, J. L. (2021). *Real-Time Web Attack Detection Using Deep Learning: Opportunities and Challenges*. This paper highlights the importance of real-time detection systems and their limitations.
5. TensorFlow Documentation. Available at: https://www.tensorflow.org
   Used for building and training deep learning models.
6. Scikit-learn Documentation. Available at: https://scikit-learn.org
   Used for implementing machine learning algorithms like SVM and Naïve Bayes.
7. NumPy Documentation. Available at: https://numpy.org
   Used for handling arrays and numerical computations.
8. Pandas Documentation. Available at: https://pandas.pydata.org
   Used for data preprocessing and analysis.
9. Matplotlib Documentation. Available at: https://matplotlib.org
   Used for visualization of results and graphs.
10. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
    A fundamental book explaining deep learning concepts and architectures.
11. Stallings, W. (2017). *Network Security Essentials: Applications and Standards*. Pearson.
    Provides knowledge about network security concepts and attack types.